



UNIVERSITÀ
DI SIENA
1240

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE
E SCIENZE MATEMATICHE

Corso di Laurea Triennale in
Ingegneria Informatica e dell'Informazione

Utilizzo delle AI nella programmazione Software

Relatore:
Prof. Marco Maggini

Candidato:
Alessandro Franco

Anno Accademico 2024-2025

Indice

1	Introduzione	2
1.1	Obiettivo	2
1.2	Strumenti Utilizzati	3
1.3	Premessa: contesto e scalabilità del caso di studio	4
1.4	Requisiti pratici del progetto	5
2	Analisi comparativa dei prompt nella progettazione assistita da AI	6
2.1	Prompt minimale	6
2.2	Analisi codici generati da un prompt minimale	9
2.3	Utilizzo del Prompt Engineering per la progettazione iniziale del sistema	9
2.4	Analisi dei documenti di progettazione generati	11
3	Fattori chiave per l'utilizzo efficace dell'AI nello sviluppo software	14
3.1	Utilizzo di prompt efficaci	14
3.2	Organizzazione del contesto di progetto	14
3.3	Integrazione dell'AI nell'ambiente di sviluppo	15
	Bibliografia	16

1. Introduzione

Negli ultimi anni l'evoluzione dei Large Language Models (LLM) ha reso possibile l'utilizzo di sistemi di intelligenza artificiale in grado di assistere attivamente il processo di sviluppo software. Strumenti come GitHub Copilot, ChatGPT, CodeWhisperer e altri sistemi basati su modelli linguistici sono oggi in grado di generare codice, correggere errori e supportare la progettazione di componenti software a partire da una descrizione del problema.

Queste capacità stanno progressivamente modificando il modo in cui gli sviluppatori affrontano il processo di programmazione. Tradizionalmente, lo sviluppo software consiste nella progettazione di un algoritmo e nella sua traduzione in un linguaggio di programmazione formale. Con l'introduzione dei modelli linguistici, invece, parte di questo processo può essere delegata a sistemi automatici capaci di generare codice a partire da descrizioni ad alto livello. L'utilizzo dell'intelligenza artificiale nella programmazione software introduce la necessità di ripensare il workflow di sviluppo a partire dalla analisi del problema fino al debugging e modifica del codice.

In questo contesto, uno degli aspetti centrali diventa la capacità di comunicare efficacemente con il modello linguistico, fornendo descrizioni chiare del problema, del contesto e dei vincoli richiesti dalla soluzione. Tuttavia, l'interazione con questi sistemi non può essere ridotta alla semplice formulazione di una richiesta testuale. Per ottenere risultati affidabili è necessario integrare l'intelligenza artificiale all'interno di un workflow di sviluppo strutturato, in cui il modello possa accedere alle informazioni rilevanti del progetto e operare all'interno di un contesto ben definito.

1.1 Obiettivo

Per analizzare concretamente queste dinamiche, il presente lavoro utilizza come caso di studio lo sviluppo reale di un'applicazione web commissionata per l'accettazione e la gestione degli ordini di una pizzeria. La realizzazione di questo sistema informatico non rappresenta quindi solamente l'obiettivo applicativo del progetto, ma costituisce anche il contesto sperimentale attraverso cui osservare e valutare il ruolo degli strumenti di intelligenza artificiale nello sviluppo del software.

Nei seguenti capitoli verranno illustrate diverse modalità di utilizzo dei sistemi di intelligenza artificiale durante il processo di sviluppo, analizzando in particola-

re come questi strumenti possano supportare lo sviluppatore nelle varie fasi della progettazione e dell'implementazione del sistema. Verranno inoltre presentati diversi casi di ottimizzazione del processo di sviluppo ottenuti attraverso l'utilizzo di ambienti di lavoro controllati, in cui l'interazione tra sviluppatore e modello linguistico è stata strutturata per massimizzare l'efficacia del supporto fornito dall'intelligenza artificiale.

Attraverso questo caso di studio sarà quindi possibile osservare come l'integrazione di strumenti basati su modelli linguistici possa influenzare il processo di sviluppo software, evidenziando sia i vantaggi operativi sia le criticità legate all'utilizzo di questi sistemi. L'obiettivo non è solamente dimostrare la capacità dei modelli di generare codice, ma analizzare in che modo tali strumenti possano essere utilizzati come supporto efficace all'interno di un processo di sviluppo strutturato.

1.2 Strumenti Utilizzati

Durante lo sviluppo dell'applicazione sono stati utilizzati diversi strumenti basati su modelli di intelligenza artificiale, integrati all'interno dell'ambiente di sviluppo con l'obiettivo di supportare le varie fasi del processo di programmazione. Questi strumenti sono stati impiegati per assistere lo sviluppatore nella progettazione dell'interfaccia utente, nella generazione e modifica del codice e nell'organizzazione della struttura del progetto.

GitHub Copilot GitHub Copilot è un assistente di programmazione basato su intelligenza artificiale integrato direttamente nell'ambiente di sviluppo Visual Studio Code. Il sistema è stato utilizzato principalmente per fornire suggerimenti automatici di codice durante la scrittura dei file sorgente. Copilot analizza il contesto del codice e propone automaticamente completamenti e implementazioni di funzioni, contribuendo a velocizzare la scrittura del codice e ridurre operazioni ripetitive durante lo sviluppo.

Google Stitch Google Stitch è uno strumento sperimentale di progettazione di interfacce utente basato su intelligenza artificiale sviluppato da Google Labs. Il sistema permette di generare prototipi di interfacce web e mobile a partire da descrizioni testuali o da semplici wireframe. Nel contesto del progetto, Stitch è stato utilizzato come strumento di supporto alla fase iniziale di progettazione dell'interfaccia grafica dell'applicazione. Attraverso l'utilizzo di descrizioni testuali è stato possibile generare rapidamente layout preliminari dell'interfaccia utente, successivamente adattati e integrati nel progetto finale.

Windsurf Windsurf è un ambiente di sviluppo integrato (IDE) progettato per integrare profondamente l'intelligenza artificiale nel processo di programmazione. A differenza dei tradizionali editor di codice, Windsurf è stato progettato come un

editor “AI-native”, in cui i modelli linguistici partecipano attivamente alla generazione, modifica e debugging del codice. Questo strumento consente di interagire con l’intera codebase attraverso comandi in linguaggio naturale, permettendo al sistema di comprendere il contesto del progetto e di generare modifiche su più file in modo coordinato. Durante lo sviluppo del progetto, Windsurf è stato utilizzato per sperimentare modalità di sviluppo assistito da AI più avanzate rispetto ai tradizionali sistemi di completamento automatico del codice.

1.3 Premessa: contesto e scalabilità del caso di studio

Il sistema software analizzato in questo lavoro consiste in un’applicazione web per la gestione degli ordini e dell’organizzazione operativa di una pizzeria. L’applicazione è progettata come sistema informativo per attività di ristorazione di piccole-medie dimensioni e presenta una struttura architetturale centralizzata, basata su una singola istanza server e su un database locale SQLite. L’interfaccia utente è accessibile tramite browser web ed è pensata per essere utilizzata da un numero limitato di operatori simultanei.

È importante sottolineare che il dominio applicativo analizzato presenta un livello di complessità architetturale relativamente contenuto. Il sistema non include infatti alcune delle caratteristiche tipiche delle applicazioni enterprise di larga scala, come architetture distribuite basate su microservizi, sistemi di messaging asincrono, gestione avanzata della concorrenza o infrastrutture di scalabilità orizzontale. Analogamente, il numero limitato di utenti simultanei e la natura prevedibile dei dati trattati rendono il modello relativamente semplice.

La scelta di un sistema con complessità moderata consente di analizzare in modo più chiaro le modalità di integrazione degli strumenti basati su intelligenza artificiale nel processo di sviluppo software. In un contesto fortemente complesso, infatti, numerosi fattori architetturali e infrastrutturali potrebbero rendere più difficile isolare e osservare gli effetti dell’utilizzo dei modelli linguistici nel workflow di programmazione.

Da notare come i pattern di sviluppo assistito da intelligenza artificiale analizzati in questo lavoro non dipendono strettamente dal dominio applicativo specifico. Tali approcci risultano infatti trasferibili anche a sistemi software di maggiore complessità, nei quali le stesse tecniche possono essere applicate su architetture più articolate.

Il caso di studio presentato in questa tesi si colloca quindi in una posizione intermedia: sufficientemente complesso da presentare problemi reali di sviluppo software, ma al tempo stesso abbastanza contenuto da permettere un’analisi completa del processo di sviluppo assistito da intelligenza artificiale.

1.4 Requisiti pratici del progetto

L'applicazione sviluppata in questo lavoro nasce da una richiesta concreta da parte del proprietario di una pizzeria, il quale necessitava di uno strumento informatico in grado di supportare sia la gestione operativa del ristorante durante il servizio sia l'analisi economica dell'attività. L'obiettivo principale era disporre di un sistema semplice da utilizzare dal personale del locale, accessibile tramite browser e in grado di centralizzare la gestione degli ordini e delle informazioni economiche.

Oltre agli aspetti operativi legati al servizio, il proprietario era soprattutto interessato alla possibilità di analizzare i dati generati dall'attività del ristorante in maniera semi automatica, in particolare, il sistema doveva essere in grado di raccogliere informazioni sugli ordini effettuati e trasformarle in indicatori utili alla gestione economica del locale. Tra le funzionalità richieste rientravano la possibilità di monitorare i ricavi generati dal ristorante, analizzare il numero di clienti serviti in determinati intervalli temporali e valutare l'andamento dei prezzi e dei guadagni nel tempo.

Un ulteriore aspetto di interesse riguardava la possibilità di ottenere statistiche aggregate sull'attività del ristorante. Il proprietario desiderava, ad esempio, poter stimare il guadagno medio per coperto, osservare il numero medio di persone servite durante il servizio e individuare eventuali variazioni nei ricavi o nella domanda dei prodotti. L'obiettivo era quello di utilizzare i dati raccolti dal sistema come supporto alle decisioni gestionali, permettendo una visione più chiara dell'andamento economico del locale.

2. Analisi comparativa dei prompt nella progettazione assistita da AI

Per valutare concretamente l'efficacia degli strumenti basati su modelli linguistici nella fase iniziale di progettazione del software, è stato condotto un semplice esperimento basato sulla formulazione di prompt con diversi livelli di dettaglio. L'obiettivo era osservare come la qualità delle risposte generate dal modello vari in funzione delle informazioni fornite nel prompt.

2.1 Prompt minimale

Per valutare il comportamento degli strumenti di intelligenza artificiale in assenza di istruzioni dettagliate, è stato inizialmente utilizzato un prompt estremamente generico.

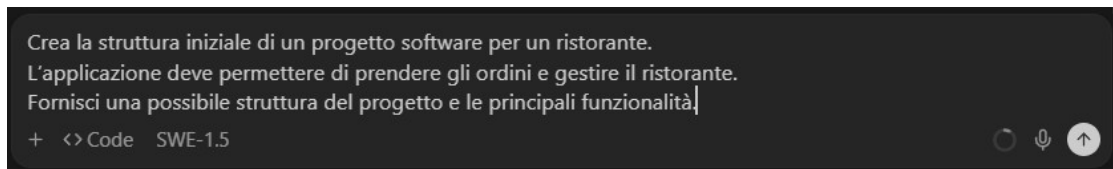


Figura 2.1: Prompt minimale utilizzato nel esperimento

Questo prompt è stato sottoposto a diversi sistemi basati su modelli linguistici di grandi dimensioni (LLM), tra cui assistenti integrati in ambienti di sviluppo e sistemi di generazione di codice.

Sebbene i risultati ottenuti dai diversi sistemi presentassero variazioni nelle tecnologie utilizzate e nell'organizzazione dei file, è emerso un comportamento comune: tutti i modelli tendevano a prendere decisioni autonome ma con profondi errori strutturali.

Per l'analisi dettagliata riportata in questa sezione è stato scelto come caso di studio il modello SWE-1.5, integrato nell'ambiente di sviluppo Windsurf. Questo sistema è stato selezionato perché la struttura di progetto generata rende particolarmente evidente l'insieme di problemi tipici derivanti dall'utilizzo di prompt troppo generici.

L'analisi del codice generato evidenzia diversi problemi strutturali che emergono

quando un modello linguistico tenta di costruire autonomamente una codebase complessa a partire da istruzioni estremamente generiche. A partire da un input molto breve, il modello ha generato automaticamente una codebase di dimensioni rilevanti, composta da 35 file e circa 3631 righe di codice ma nonostante l'apparente completezza del codice e della struttura, il programma si è rivelato fallimentare dal punto di vista strutturale e funzionale.

Una prima categoria di criticità riguarda l'incoerenza tra le tecnologie utilizzate e le strutture dati generate dal modello. In diversi punti del progetto il codice assumeva la disponibilità di costrutti o convenzioni non pienamente compatibili con il sistema di persistenza effettivamente utilizzato. Questo tipo di errore indica che il modello ha generato il codice sulla base di schemi generici appresi durante l'addestramento, senza una reale integrazione tra le scelte tecnologiche adottate nel progetto.

```
CREATE TABLE IF NOT EXISTS users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT UNIQUE NOT NULL,  
  isActive BOOLEAN DEFAULT 1, // ✗ ERRORE: BOOLEAN non supportato  
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Figura 2.2: Esempio di incompatibilità tra struttura dei dati e sistema di persistenza

Una seconda categoria di problemi riguarda la gestione dell'inizializzazione del sistema e delle dipendenze tra le diverse componenti dell'applicazione. In più punti il codice tentava di eseguire operazioni su strutture non ancora disponibili oppure di creare relazioni tra entità non ancora definite. In sistemi che prevedono database relazionali e inizializzazione asincrona delle risorse, la corretta sequenza delle operazioni rappresenta un requisito fondamentale per la stabilità del sistema.

```
const createTables = () => {  
  db.run('CREATE TABLE users...'); // ✗ ERRORE: non aspetta il completamento  
  db.run('CREATE TABLE menu_items...'); // ✗ ERRORE: esegue prima che users sia creata  
  console.log('Tables created successfully');  
};
```

Figura 2.3: Esempio di errore logico dovuto alla gestione non corretta delle dipendenze tra componenti

Anche dopo la correzione manuale di questi problemi iniziali, il progetto continuava a presentare numerosi difetti strutturali. Il codice risultava caratterizzato da funzioni duplicate, componenti ridondanti e una struttura interna poco coerente. Questo suggerisce che il modello aveva generato una grande quantità di codice

senza una reale comprensione dello scope del sistema da realizzare.
Dal punto di vista dell'interfaccia utente, il risultato era altrettanto limitato.

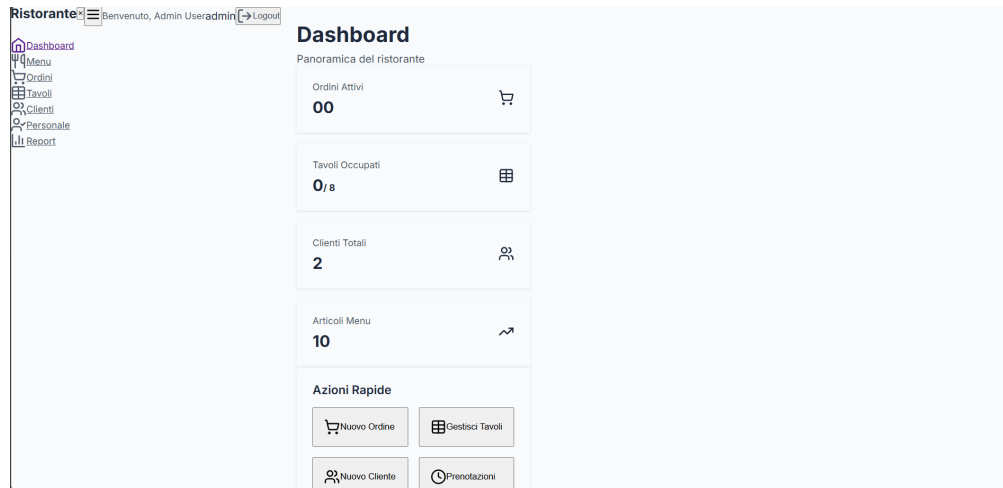


Figura 2.4: Interfaccia generata automaticamente dal modello

Sebbene l'applicazione apparisse inizialmente funzionante, ogni tentativo di interazione con l'interfaccia produceva comportamenti inattesi oppure errori di esecuzione. Questo indica che il problema non era solamente legato alla qualità estetica dell'interfaccia, ma alla stabilità complessiva del sistema generato.

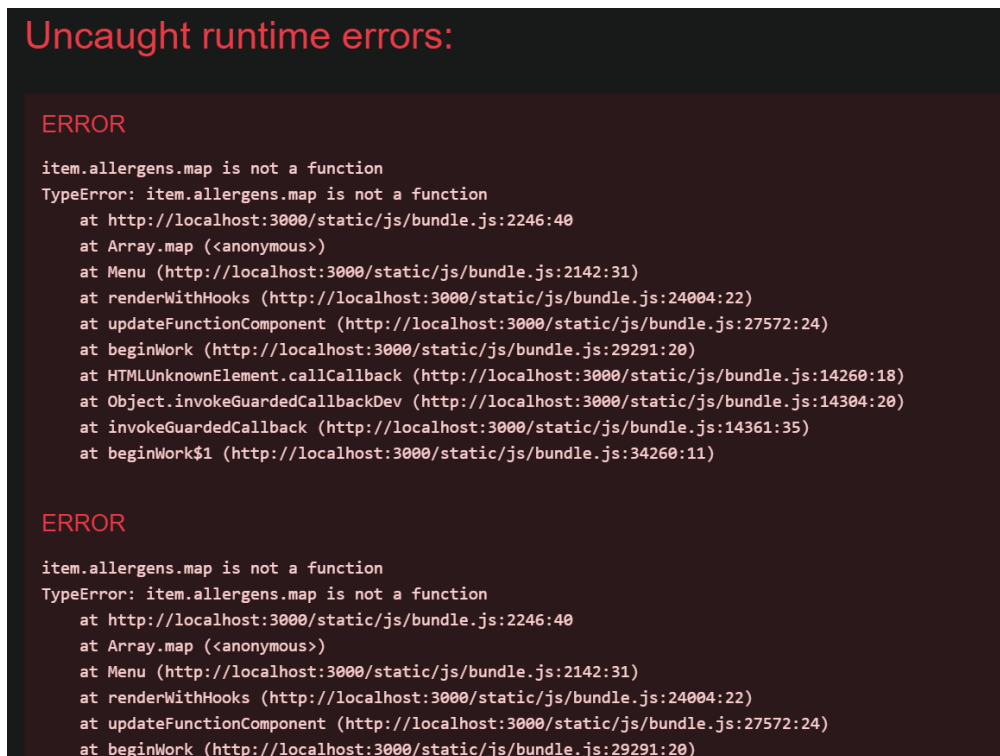


Figura 2.5: Esempio di errore di runtime generato dall'applicazione

2.2 Analisi codici generati da un prompt minimale

Nel complesso, questo primo esperimento evidenzia un aspetto centrale dell'utilizzo dei modelli linguistici nello sviluppo software: in assenza di vincoli progettuali sufficientemente espliciti, il modello tende a generare soluzioni estese ma fragili, introducendo autonomamente decisioni architetturali non richieste e spesso non adatte al contesto applicativo.

Il modello linguistico può quindi produrre rapidamente grandi quantità di codice anche partendo da istruzioni minime; tuttavia questa apparente produttività non coincide necessariamente con una reale efficacia progettuale. In mancanza di una guida precisa, l'AI rischia infatti di aumentare il carico di revisione dello sviluppatore, che si trova costretto a correggere errori strutturali, incoerenze architetturali e bug funzionali distribuiti lungo tutta la codebase.

Questo comportamento non deve tuttavia essere interpretato esclusivamente come un limite della tecnologia. L'esperimento mostra infatti che, in presenza di specifiche incomplete, il modello tende a generare autonomamente ipotesi progettuali per colmare le informazioni mancanti. Le soluzioni prodotte risultano spesso plausibili dal punto di vista formale, ma non necessariamente coerenti sul piano architetturale.

Questa proprietà rappresenta allo stesso tempo un limite e un'opportunità. Da un lato può introdurre errori strutturali quando il contesto applicativo non è definito in modo sufficientemente preciso; dall'altro consente di ottenere rapidamente una prima struttura di soluzione che può essere successivamente raffinata e corretta dallo sviluppatore.

Per ridurre questo tipo di ambiguità è necessario adottare metodologie più strutturate nella definizione delle richieste rivolte ai modelli linguistici. In questo contesto assume particolare rilevanza il concetto di *Prompt Engineering*, inteso come la progettazione sistematica dei prompt con l'obiettivo di guidare il comportamento del modello e ottenere risultati maggiormente controllabili e coerenti dal punto di vista ingegneristico.

Nel seguito verrà quindi analizzato come la riformulazione del prompt mediante tecniche di Prompt Engineering consenta di mitigare i problemi osservati in questa prima fase sperimentale e di migliorare la qualità delle soluzioni generate.

2.3 Utilizzo del Prompt Engineering per la progettazione iniziale del sistema

A seguito dei risultati ottenuti con il prompt minimale, è stato introdotto un prompt più strutturato con l'obiettivo di guidare in maniera più precisa il comportamento del modello linguistico. Questa tecnica prende il nome di *Prompt Engineering* ed è definita come il processo di progettazione e ottimizzazione degli input testuali forniti ai modelli linguistici al fine di ottenere risposte più accurate, controllabili e coerenti con l'obiettivo del task [2].

Diversi studi evidenziano infatti che la qualità dell'output generato dai Large Language Models (LLM) dipende in maniera significativa dalla struttura e dalla specificità del prompt utilizzato. In particolare, fornire contesto esplicito, vincoli progettuali e una chiara definizione del formato di output riduce l'ambiguità e migliora la qualità delle soluzioni generate [3].

Nel contesto di questo lavoro, il prompt utilizzato è stato progettato suddividendo le informazioni in diverse sezioni logiche:

definizione del task

- contesto applicativo
- richiesta progettuale
- requisiti tecnici
- vincoli di progettazione
- elementi da evitare (negative prompt)
- formato dell'output richiesto

Questo approccio segue delle raccomandazioni secondo cui i modelli linguistici tendono a produrre risultati più coerenti quando il problema viene esplicitamente decomposto in sotto-componenti verificabili [1].

prompt utilizzato per generare la struttura iniziale dell'applicazione

Task: Progettare la struttura iniziale di un'applicazione web per la gestione di un ristorante, preparando documenti di testo txt e markdown.

Context: L'applicazione è commissionata dal proprietario di una pizzeria.

Il sistema deve permettere: presa ordini da parte dei camerieri, gestione tavoli, gestione menu, analisi economica del ristorante

Utenti previsti: cameriere, manager, proprietario

Request: Proporre una struttura architeturale iniziale dell'applicazione includendo: struttura del progetto, componenti principali del sistema, schema del database, principali API, flusso dei dati tra interfaccia, backend e database

Technical Requirements: web application accessibile tramite browser, backend realizzato con Python FastAPI, separazione tra frontend e backend, utilizzo di database, gestione dinamica del menu, tracciamento storico degli ordini, supporto ad analisi economiche

Constraints: Sistema destinato a ristorante di piccole dimensioni. Non è necessario progettare infrastruttura distribuita o cloud.

Negative Prompt: evitare architetture enterprise complesse, evitare microservizi, evitare menu hard-coded, evitare soluzioni senza database

Rispetto all'esperimento precedente, il modello non ha prodotto direttamente grandi quantità di codice applicativo, ma ha generato una serie di documenti di progettazione che descrivono l'architettura del sistema, la struttura del progetto, lo schema preliminare del database e i principali moduli applicativi.

2.4 Analisi dei documenti di progettazione generati

L'esecuzione del prompt strutturato ha portato alla generazione di una serie di documenti di progettazione che descrivono in maniera preliminare l'architettura del sistema, il flusso dei dati tra i componenti applicativi, lo schema del database e la struttura organizzativa del progetto.

A differenza del caso precedente, in cui il modello tendeva a generare direttamente porzioni di codice difficilmente verificabili, l'output prodotto in questa fase assume la forma di una documentazione architeturale preliminare. Questa modalità di generazione risulta particolarmente utile nel contesto della progettazione software, poiché consente allo sviluppatore di analizzare e validare le scelte strutturali prima di procedere con l'implementazione effettiva del sistema.

Architettura del sistema

Il documento relativo all'architettura del sistema descrive un'architettura applicativa a tre livelli composta da interfaccia utente, backend applicativo e database. In particolare, il modello propone un'architettura client-server nella quale il frontend è costituito da una interfaccia web accessibile tramite browser, mentre il backend espone una serie di API REST realizzate tramite il framework FastAPI.

Il database proposto è di tipo relazionale e viene inizialmente implementato tramite SQLite, con la possibilità di migrare successivamente verso soluzioni più scalabili come PostgreSQL. L'architettura suggerita include inoltre l'utilizzo di un ORM per la gestione delle entità persistenti e di meccanismi di autenticazione basati su token.

Il sistema prevede tre tipologie principali di utenti: camerieri, manager e proprietario, ciascuno dotato di differenti livelli di accesso alle funzionalità applicative. Questa suddivisione introduce un modello di controllo degli accessi basato su ruoli (Role-Based Access Control), frequentemente utilizzato nelle applicazioni gestionali.

Flusso dei dati

Il documento relativo al flusso dei dati descrive il ciclo di vita delle principali operazioni del sistema. In particolare, viene rappresentato il percorso seguito dalle richieste dell'utente dal momento in cui vengono generate nel frontend fino alla loro elaborazione nel backend e alla successiva persistenza nel database.

Il modello descrive un flusso dati tipico nel quale il browser invia richieste HTTP al backend tramite API REST, il backend esegue la logica applicativa e interagisce con il database per la lettura o la scrittura dei dati, e infine restituisce una risposta al client. In alcune operazioni, come la creazione di nuovi ordini o l'aggiornamento dello stato dei tavoli, viene inoltre suggerito l'utilizzo di comunicazioni real-time tramite WebSocket per sincronizzare le interfacce dei diversi operatori del ristorante.

Schema del database

Un ulteriore documento generato dal modello riguarda la definizione preliminare dello schema del database. Lo schema proposto include diverse tabelle che rappresentano le principali entità del dominio applicativo del ristorante, tra cui utenti, tavoli, categorie del menu, prodotti del menu, ordini e pagamenti.

Le relazioni tra queste entità sono modellate tramite chiavi esterne che consentono di rappresentare i legami logici tra gli elementi del sistema. Ad esempio, ogni ordine è associato a un tavolo e a un cameriere, mentre i singoli prodotti ordinati sono rappresentati tramite una tabella di dettaglio collegata sia all'ordine sia agli elementi del menu.

Questo schema segue una struttura relazionale normalizzata che permette di evitare ridondanze nei dati e facilita la realizzazione di interrogazioni analitiche relative all'andamento economico del ristorante.

Moduli applicativi e API

Il modello propone inoltre una suddivisione modulare del backend in diversi componenti funzionali. Tra i moduli principali individuati vi sono il modulo di autenticazione, il modulo di gestione utenti, il modulo di gestione dei tavoli, il modulo di gestione del menu, il modulo di gestione degli ordini e un modulo dedicato all'analisi dei dati.

Ciascun modulo espone un insieme di endpoint API dedicati alle operazioni di lettura, creazione, aggiornamento e cancellazione delle entità del sistema. Questa organizzazione favorisce una separazione chiara tra le diverse responsabilità applicative e rende il codice più facilmente manutenibile e scalabile.

Struttura del progetto

Infine, il modello genera anche una proposta di struttura organizzativa del progetto software. La struttura suggerita prevede una separazione tra backend e frontend, con una suddivisione interna del backend in moduli dedicati ai modelli dati, agli schemi di validazione, alle route API e alla logica applicativa.

L'analisi complessiva dei documenti generati mostra come il modello sia in grado di produrre una struttura progettuale coerente con i requisiti espressi nel prompt. In particolare, l'architettura proposta rispetta i principali vincoli tecnici specificati, come l'utilizzo di FastAPI per il backend e la separazione tra interfaccia utente,

logica applicativa e database.

Tuttavia, durante la fase di implementazione pratica del sistema sono emerse alcune criticità. In particolare, il modello tendeva frequentemente a proporre soluzioni architetturali più complesse rispetto ai requisiti effettivamente necessari, introducendo componenti o configurazioni aggiuntive non richieste dal prompt. Questo comportamento porta spesso ad un aumento dello scope del progetto e a forme di overengineering, nelle quali vengono suggerite funzionalità o strutture pensate per applicazioni più complesse rispetto al caso di studio considerato.

Questo fenomeno è coerente con quanto osservato nella letteratura sui Large Language Models applicati allo sviluppo software: in presenza di specifiche parziali o non completamente vincolate, il modello tende a completare il progetto facendo riferimento a pattern architetturali comuni o a soluzioni considerate best practice. L'analisi dei documenti prodotti mostra una discreta coerenza con i requisiti espressi nel prompt. Il modello propone infatti un'architettura a tre livelli composta da interfaccia utente, backend applicativo e database, oltre a identificare le principali entità del dominio applicativo del ristorante, come ordini, tavoli, prodotti del menu e utenti.

Questo esperimento conferma quindi un aspetto fondamentale dell'utilizzo dei LLM nello sviluppo software: la qualità dell'output non dipende esclusivamente dalle capacità del modello, ma in larga misura dalla qualità e dalla struttura del prompt utilizzato per guidarne il comportamento.

3. Fattori chiave per l'utilizzo efficace dell'AI nello sviluppo software

L'utilizzo di modelli di intelligenza artificiale all'interno del processo di sviluppo software non si limita alla semplice generazione di codice tramite prompt. Durante lo sviluppo dell'applicazione realizzata in questa tesi è emerso come l'efficacia di tali strumenti dipenda fortemente da tre aspetti principali: la formulazione dei prompt, l'organizzazione del contesto di progetto fornito al modello e l'integrazione dell'AI all'interno di un ambiente di sviluppo adeguato.

3.1 Utilizzo di prompt efficaci

Il primo fattore riguarda la formulazione dei prompt utilizzati per interagire con il modello. Prompt generici tendono infatti a produrre output poco strutturati o non perfettamente coerenti con i requisiti del progetto. Al contrario, prompt più dettagliati e contestualizzati permettono di guidare il modello verso soluzioni più precise e aderenti alle necessità dello sviluppatore.

Nel corso dello sviluppo dell'applicazione si è osservato come la qualità del codice generato dipenda fortemente dalla chiarezza con cui vengono specificati i vincoli architetturali, le tecnologie utilizzate e il contesto applicativo. In particolare, la presenza di indicazioni esplicite riguardo al framework utilizzato, alla struttura del progetto e alle funzionalità richieste permette al modello di ridurre le ambiguità e produrre soluzioni più coerenti.

3.2 Organizzazione del contesto di progetto

Un secondo aspetto fondamentale riguarda l'organizzazione delle informazioni che vengono rese disponibili al modello durante l'interazione. Nel progetto sviluppato in questa tesi sono stati introdotti diversi file di documentazione testuale, come file Markdown e file di configurazione, contenenti descrizioni dell'architettura del sistema, della struttura del database e delle principali funzionalità dell'applicazio-

ne.

Questi documenti permettono di fornire al modello un contesto più ampio rispetto al singolo file di codice su cui si sta lavorando, migliorando la coerenza delle risposte generate. In pratica tali file funzionano come una forma di memoria strutturata del progetto, consentendo all'intelligenza artificiale di comprendere meglio l'organizzazione del sistema e di adattare le proprie proposte alle scelte progettuali già effettuate.

3.3 Integrazione dell'AI nell'ambiente di sviluppo

Il terzo fattore riguarda l'integrazione degli strumenti di intelligenza artificiale all'interno dell'ambiente di sviluppo. L'utilizzo di editor e piattaforme che integrano direttamente modelli di AI consente di rendere il processo di sviluppo più fluido, permettendo allo sviluppatore di alternare rapidamente la scrittura del codice, la generazione assistita di nuove funzionalità e l'analisi di eventuali errori.

In questo contesto risultano particolarmente utili strumenti dedicati al debugging e al testing, che permettono di verificare rapidamente il comportamento del codice generato. L'integrazione tra AI e strumenti di sviluppo tradizionali consente quindi di utilizzare il modello non solo per la generazione iniziale del codice, ma anche come supporto nelle fasi di revisione, correzione e miglioramento del software.

Bibliografia

- [1] Marco Midolo et al. Guidelines to prompt large language models for code generation. *arXiv preprint arXiv:2601.13118*, 2026.
- [2] Pranab Sahoo et al. A systematic survey of prompt engineering in large language models. *arXiv preprint arXiv:2402.07927*, 2024.
- [3] Jules White et al. Chatgpt prompt patterns for improving code quality. *arXiv preprint arXiv:2303.07839*, 2023.